Easy Peasy — TryHackMe (Fácil)

Plataforma: TryHackMe **Dificultad:** Fácil **Categorías:** Reconocimiento de red, enumeración web, criptografía básica, esteganografía, escalada de privilegios en Linux **Tiempo invertido:** ~5 horas, distribuidas en 3 días **Herramientas:** Nmap, Gobuster, curl, CyberChef, John the Ripper, steghide, SSH

Índice

1. Resumen
2. Reconocimiento
3. Enumeración web
4. Contenido oculto en el código fuente
5. Hash y cracking
6. Esteganografía
7. Acceso por SSH
8. Escalada de privilegios
9. Lecciones aprendidas
10. Mitigaciones
11. Referencias

Resumen

Easy Peasy es una máquina Linux con dos servidores web, un servicio SSH en puerto no estándar, y una cadena de pistas progresivas (texto oculto, codificaciones, un hash poco común, esteganografía en una imagen) que termina en una escalada de privilegios real mediante una tarea cron mal configurada.

1. Reconocimiento

Escaneo completo de puertos, porque un escaneo por defecto solo revisa los 1000 puertos más comunes y aquí uno de los servicios usaba un puerto alto:

```
nmap -p- <IP>
```

Resultado: 3 puertos abiertos.

```
nmap -sV -p 80,6498,65524 <IP>
```

Puerto	Servicio	Detalle
80	http	nginx
6498	ssh	OpenSSH
65524	http	Apache

SSH en un puerto fuera del estándar (22) es una práctica común para reducir el ruido de escaneos automáticos, aunque no resiste un escaneo completo de puertos como el de arriba.

2. Enumeración web

Enumeración de directorios con Gobuster sobre ambos servidores:

```
gobuster dir -u http://<IP>:65524 -w /usr/share/wordlists/dirb/common.txt
```

En el nginx (puerto 80), `robots.txt` contenía un hash de 32 caracteres hexadecimales. Por su longitud, la hipótesis inicial fue MD5, confirmada con `md5sum`.

Con la lista `common.txt` no apareció todo: en el Apache, un segundo Gobuster contra un subdirectorio, esta vez con la lista `big.txt`, reveló una carpeta adicional que la lista corta no había encontrado. Lección: una lista de palabras corta puede dar una falsa sensación de que "no hay nada más".

3. Contenido oculto en el código fuente

El navegador no mostraba nada en varias páginas, pero el HTML sí tenía contenido: texto marcado con la etiqueta `hidden`, que un navegador respeta pero que sigue presente en el código.

```
curl -s http://<IP>:PUERTO/ | grep -i hidden
```

Un hallazgo decía "encoded with ba....", seguido de una cadena alfanumérica. Por el conjunto de caracteres (sin símbolos `+`, `/` ni `=`) y comprobando que el resultado decodificado coincidía en longitud con el formato de respuesta esperado por la sala, se identificó como **base62** en lugar del más común base64, y se decodificó con CyberChef (`From Base62`).

Otra flag apareció directamente en el HTML de una de las páginas, visible solo leyendo el código fuente completo, no la página renderizada.

4. Hash y cracking

Un hash de 64 caracteres apareció en otra página. El supuesto inicial, por longitud, fue SHA-256 — resultó ser **incorrecto**. `rockyou.txt` completo (14M+ entradas) no produjo ningún resultado contra el formato SHA-256, lo cual era en sí mismo la señal de que el formato estaba mal identificado.

La propia tarea de la sala especificaba el algoritmo real: **GOST** (GOST R 34.11-94), un algoritmo que coincide en longitud de salida con SHA-256 pero es completamente distinto.

```
john --format=gost --wordlist=<wordlist_de_la_sala> hash.txt
```

Lección clave: la longitud de un hash es una pista, no una prueba. Dos algoritmos distintos pueden producir salidas del mismo tamaño.

5. Esteganografía

Una imagen en una de las páginas sugería contenido oculto. Se descargó y se analizó con steghide, usando la contraseña recuperada del hash GOST:

```
wget http://<IP>:65524/<ruta>/imagen.jpg
steghide extract -sf imagen.jpg -p <contraseña>
```

El archivo extraído contenía un usuario y una contraseña en **binario**, decodificados con CyberChef (`From Binary`).

6. Acceso por SSH

```
ssh -p 6498 <usuario>@<IP>
```

Dentro de la máquina, el archivo de la flag de usuario tenía su contenido en **ROT13** (un cifrado César de desplazamiento 13), con una pista explícita en el propio archivo ("seems wrong, like it's rotated"). Se decodificó con CyberChef (`ROT13`).

7. Escalada de privilegios

Camino explorado: se identificó CUPS 2.2.7 corriendo localmente en el puerto 631 (solo accesible desde dentro de la máquina, no visible en el escaneo externo). La investigación llevó a identificar la vulnerabilidad asociada a esa versión exacta (CVE-2018-16860), un desbordamiento de búfer en la pila que requiere desarrollo de exploit a medida (shellcode) para convertirse en ejecución de código. Esa técnica está fuera del alcance de mis conocimientos actuales en explotación binaria, así que, en lugar de intentarla a ciegas, prioricé buscar un vector de escalada más accesible — decisión razonable dado que la sala es de nivel introductorio.

El vector real:

```
sudo -l          # sin permisos sudo para el usuario
find / -perm -4000 -type f 2>/dev/null   # sin binarios SUID inusuales
find / -writable -type f 2>/dev/null | grep -v "^/proc"
```

Este último comando reveló un script dentro de `/var/www/` con permiso de escritura para cualquier usuario. Su contenido confirmaba que se ejecutaba como root:

```
grep -r "<nombre_del_script>" /etc/crontab /etc/cron.d/
# * * * * * root cd /var/www/ && sudo bash <nombre_del_script>
```

Una tarea cron ejecutaba este script como root cada minuto. Al tener permiso de escritura sobre un script que root ejecuta automáticamente, es posible inyectar comandos arbitrarios que correrán con privilegios de root en la siguiente ejecución.

Primer intento: copiar el contenido de la flag de root a un archivo en `/tmp` con permisos abiertos. Este intento falló — el archivo resultante aparecía vacío, probablemente por una condición de carrera entre la edición del script y una ejecución del cron a mitad de escritura.

Enfoque alternativo, más directo: en vez de copiar el contenido a un archivo intermedio, modificar los permisos del archivo original y de su carpeta contenedora:

```
echo 'chmod 755 /root; chmod 644 /root/<archivo_flag>' >> <script_cron>
```

Tras la siguiente ejecución del cron, un intento de lectura directa falló con "No such file or directory" — el archivo real resultó tener un punto inicial que lo marcaba como oculto en el sistema, algo que no había considerado en el primer intento. Corregido el nombre, la lectura directa confirmó la escalada de privilegios.

Lecciones aprendidas

- La longitud de un hash no determina su algoritmo; hay que confirmarlo antes de invertir tiempo en cracking.
- Una lista de palabras corta puede ocultar resultados que una lista más grande sí encuentra.
- El contenido oculto en HTML (etiquetas `hidden`, comentarios) es invisible en el navegador pero completamente legible con `curl`.
- Un script escribible ejecutado por una tarea cron con privilegios elevados es un vector de escalada directo y de bajo esfuerzo — conviene revisarlo antes que vulnerabilidades más complejas del software instalado.
- Modificar permisos del recurso original puede ser más directo y confiable que copiar su contenido a una ubicación intermedia.
- Verificar nombres de archivo exactos (incluyendo archivos ocultos) antes de concluir que algo "no existe".

Mitigaciones (perspectiva defensiva)

- No dejar scripts ejecutados por root con permisos de escritura para otros usuarios.
- Revisar periódicamente los permisos de archivos referenciados en tareas cron del sistema.
- No exponer información en comentarios, atributos `hidden` o metadatos de páginas web.
- Usar algoritmos de hash modernos con sal (no GOST ni MD5) para almacenamiento de contraseñas.

Referencias

- TryHackMe — Easy Peasy
- CyberChef
- GTFOBins — Linux binaries that can bypass local security restrictions
- CVE-2018-16860 — CUPS local privilege escalation (NVD)